

Message Passing

Interprocess communication without shared memory

Definition

Message passing is an interprocess communication method in which processes exchange information by explicitly sending and receiving messages.

Unlike monitors, message passing is not mainly a language construct. The basic operations are usually system calls, and they can be wrapped in library procedures.

1. Basic Primitives

Two Main Operations

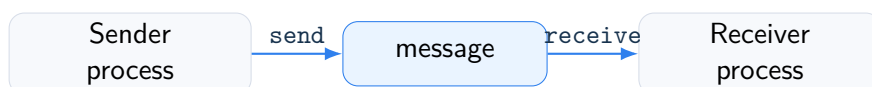
Message passing uses two basic primitives:

- `send(destination, &message)`
- `receive(source, &message)`

Primitive	Meaning
<code>send(destination, &message)</code>	Send a message to a specified destination.
<code>receive(source, &message)</code>	Receive a message from a specified source, or from ANY if the receiver does not care who sent it.

Blocking Receive

If no suitable message is available, **receive** may block until one arrives. Another design is for **receive** to return immediately with an error code.



2. Why Message Passing Is Needed

Main Idea

Semaphores and monitors assume shared memory. Message passing can work even when processes are on different machines and do not share an address space.

Distributed Communication

When two computers communicate over a network, one process cannot simply read or update the other process's memory. Information must be sent as messages.

3. Design Issues in Message-Passing Systems

Extra Problems

Message passing has design problems that do not arise in the same way with semaphores or monitors. These problems become especially important when messages travel across a network.

Issue	Meaning
Lost messages	A message may disappear because of network failure or congestion.
Duplicate messages	A retransmitted message may arrive even though the original already arrived.
Naming	The system must identify the destination of send and the source of receive unambiguously.
Authentication	A process may need proof that it is communicating with the correct service.
Performance	Copying messages is usually slower than using shared-memory synchronization.

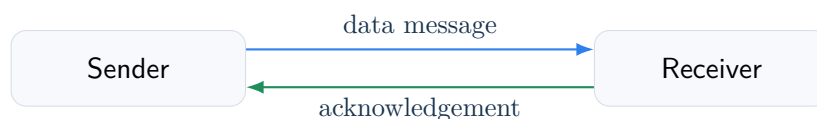
4. Lost Messages and Acknowledgements

Acknowledgement

An **acknowledgement** is a special reply message sent by the receiver to confirm that a message arrived successfully.

Basic Reliability Strategy

The sender transmits a message and waits for an acknowledgement. If the acknowledgement does not arrive within a chosen time interval, the sender retransmits the message.



Timeout

A **timeout** is the maximum time the sender waits for an acknowledgement before assuming that something went wrong.

5. Duplicate Messages and Sequence Numbers

Lost Acknowledgement Problem

Suppose the original message arrives correctly, but the acknowledgement is lost. The sender times out and sends the same message again, so the receiver gets a duplicate.

Sequence Number

A **sequence number** is a number placed in each original message so the receiver can identify old, duplicate, or out-of-order messages.

Duplicate Detection

If the receiver gets a message with the same sequence number as the previous message, it can recognize the message as a retransmission and ignore the duplicate data.

Step	Event	Result
1	Sender sends message #12.	Receiver gets it and processes it.
2	Receiver sends acknowledgement.	The acknowledgement is lost.
3	Sender times out.	Sender retransmits message #12.
4	Receiver sees #12 again.	Receiver treats it as a duplicate.

6. Naming and Authentication

Naming Problem

The system must make sure that the destination named in **send** and the source named in **receive** are clear and unambiguous.

Authentication Problem

A client may think it is talking to a file server, but an imposter process could pretend to be that server. Message systems need ways to prove identities.

7. Performance Cost

Copying Overhead

Copying a message from one process to another is usually slower than doing a semaphore operation or entering a monitor.

Even when the sender and receiver are on the same machine, the operating system may need to copy data between address spaces or manage message buffers.

Implementation Goal

Much work in operating systems and networks focuses on making message passing efficient while preserving isolation between processes.

8. Producer-Consumer Without Shared Memory

Goal

Solve the producer-consumer problem using message passing instead of shared memory, semaphores, or monitors.

Assumptions

- All messages are the same size.
- Messages that have been sent but not yet received are buffered automatically by the operating system.
- A total of N messages is used, similar to N buffer slots.

Key Trick

The consumer starts by sending N empty messages to the producer. The producer must receive an empty message before it can send back a full message.

9. Producer-Consumer Code

```
#define N 100 /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m; /* message buffer */

    while (TRUE) {
        item = produce_item(); /* generate item */
        receive(consumer, &m); /* wait for an empty */
        build_message(&m, item); /* put item in message */
        send(consumer, &m); /* send full message */
    }
}

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++)
        send(producer, &m); /* send N empties */

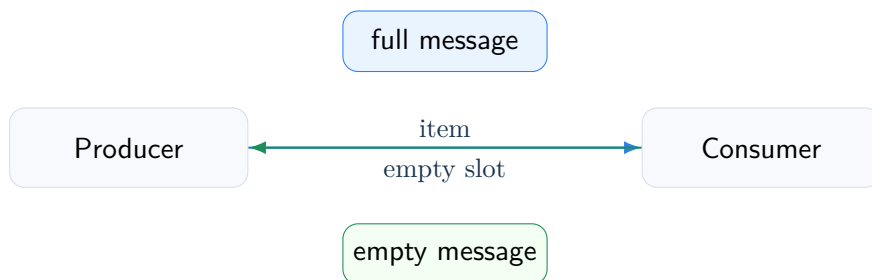
    while (TRUE) {
        receive(producer, &m); /* get full message */
        item = extract_item(&m); /* extract item */
        send(producer, &m); /* return empty */
        consume_item(item); /* use item */
    }
}
```

Constant Number of Messages

The total number of messages in the system stays constant. Messages change between empty and full, but the system never needs more than N message buffers.

10. How Blocking Happens

Situation	Result
Producer is faster	All messages become full and wait for the consumer. The producer blocks waiting for an empty message.
Consumer is faster	All messages become empty and wait for the producer. The consumer blocks waiting for a full message.



11. Direct Process Addressing

Process Names

One way to address messages is to give every process a unique address. Then **send** and **receive** name processes directly.

```
send(consumer, &message)
receive(producer, &message)
```

Naming Requirement

Direct process addressing works only if process names are unique, stable enough to use, and understood by both sides.

12. Mailboxes

Mailbox

A **mailbox** is a system-managed place where messages can be buffered. Processes send messages to a mailbox and receive messages from a mailbox.

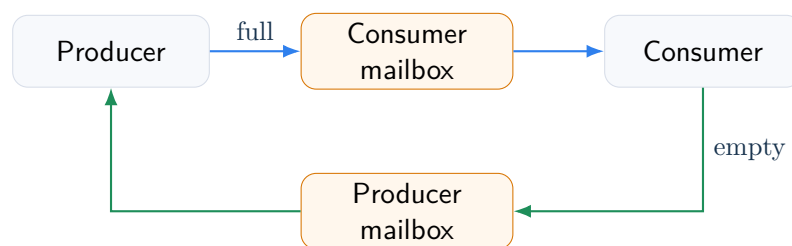
Mailbox Addressing

When mailboxes are used, the address parameters in **send** and **receive** are mailbox names instead of process names.

Mailbox Feature	Meaning
Bounded capacity	A mailbox can hold only a fixed number of messages.
Buffered messages	Messages wait in the mailbox until a receiver accepts them.
Full mailbox	A sender trying to send to a full mailbox is suspended until space becomes available.

Producer-Consumer with Mailboxes

The producer can send full messages to the consumer's mailbox, and the consumer can send empty messages to the producer's mailbox. Each mailbox can be large enough to hold N messages.



13. Rendezvous: No Buffering

Rendezvous

A **rendezvous** is a message-passing design with no buffering. The sender and receiver must meet at the same time for the message transfer to complete.

Order of Operations	Result
send happens first	The sender blocks until a matching receive happens.
receive happens first	The receiver blocks until a matching send happens.
Both are ready	The message can be copied directly from sender to receiver.

Tradeoff

Rendezvous is easier to implement than buffered message passing, but it is less flexible because the sender and receiver are forced to run in lockstep.

14. Buffered vs. Unbuffered Message Passing

Feature	Buffered Messages	Rendezvous
Storage	Messages can wait in system buffers or mailboxes.	No message buffer is kept.
Sender behavior	Sender may continue if buffer space exists.	Sender waits until receiver is ready.
Receiver behavior	Receiver may receive a message already waiting.	Receiver waits until sender is ready.
Flexibility	More flexible.	Less flexible, more synchronized.

15. Message Passing vs. Shared-Memory Synchronization

Feature	Semaphores / Monitors	Message Passing
Memory model	Usually assumes shared memory.	Works without shared memory.
Main purpose	Coordinate access to shared data.	Transfer data and synchronize processes.
Implementation	Semaphore operations or monitor entry rules.	<code>send</code> and <code>receive</code> system calls.
Distributed use	Not enough by themselves across machines.	Natural fit for networked processes.
Cost	Usually lower overhead on one machine.	Usually higher overhead due to copying and buffering.

16. MPI

Message-Passing Interface

MPI stands for Message-Passing Interface. It is a widely used message-passing system in parallel programming, especially scientific computing.

Big Idea

MPI lets programs running on many processors or machines coordinate by explicitly sending and receiving messages.

17. Big Picture

Main Conclusion

Message passing is an interprocess communication method based on `send` and `receive`. It is especially useful when processes do not share memory, but it introduces design issues such as lost messages, duplicate messages, naming, authentication, buffering, and performance.

Exam Shortcut

Remember: message passing uses **send** and **receive**; **receive** may block or return an error; lost messages need acknowledgements and timeouts; duplicates need sequence numbers; mailboxes provide buffering; rendezvous means no buffering.

18. Quick Review

Key Points

- Message passing exchanges information through explicit messages.
- The two basic primitives are **send** and **receive**.
- **receive** can block if no message is available.
- Message passing works even when processes do not share memory.
- Lost messages can be handled with acknowledgements and retransmission.
- Lost acknowledgements can create duplicate messages.
- Sequence numbers help receivers detect duplicates.
- Message systems must solve naming and authentication problems.
- Copying messages can be slower than semaphores or monitors.
- Producer-consumer can be solved by circulating N empty/full messages.
- Mailboxes are buffered destinations for messages.
- Rendezvous message passing has no buffering and forces sender and receiver to meet.
- MPI is a common message-passing system for parallel programming.